



Vietnam National University of HCMC
International University
School of Computer Science and Engineering



Object Oriented Programming

SOLID Principles

Instructor: Le Thi Ngoc Hanh, Ph.D

ltnhanh@hcmiu.edu.vn

Semester – Spring 2026

Lecture Objectives

- ☑ Recognize the relationship between SOLID principles and core OOP concepts.
- ☑ Understand how SOLID helps in Object-Oriented solution
- ☑ Identify design problems in simple class design and Apply the better solution

Introduction

SOLID

S: Single responsibility principle

O: Open/closed principle

L: Liskov substitution principle

I: Interface segregation principle

D: Dependency inversion principle

Introduction

SOLID

- S**: Single responsibility principle
- O**: Open/closed principle
- L**: Liskov substitution principle
- I**: Interface segregation principle
- D**: Dependency inversion principle



Low coupling
High cohesion
Easy to maintain
Easy to extend
Easy to test
Easy to reuse

Single responsibility principle (SRP)

A class should have only one reason to change

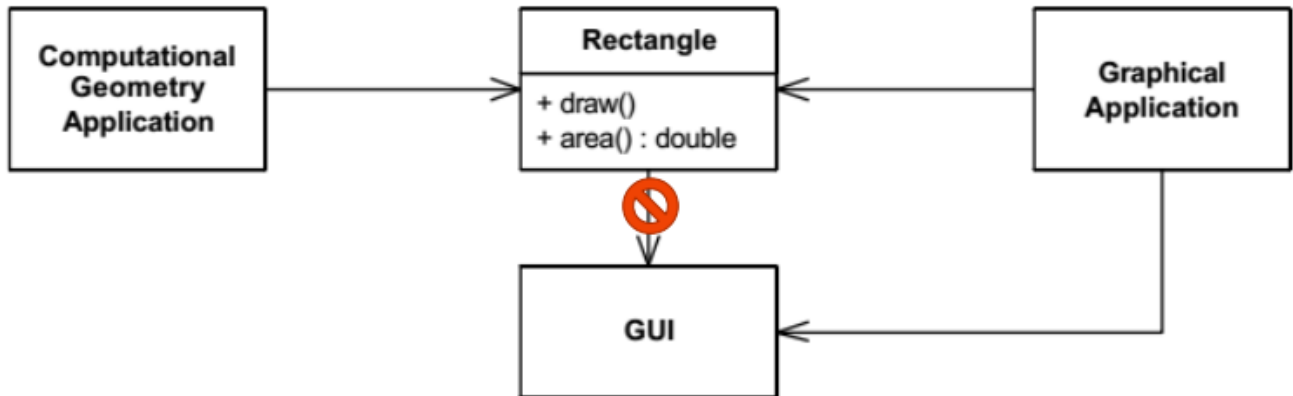
Otherwise: responsibility coupled

Changes in one responsibility may **impair or inhibit** the class's ability to meet the others.

→ Fragile design: one change may break the code in unexpected way

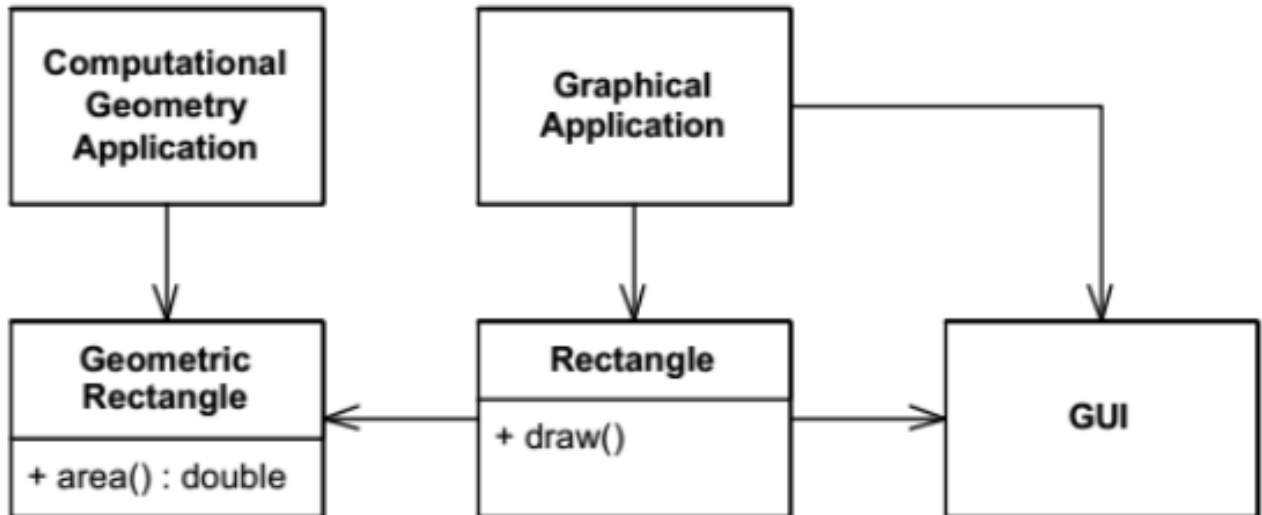
Single responsibility principle

Example – violation of SRP:



Single responsibility principle

Better model:



Single responsibility principle

Conclusion on SRP:

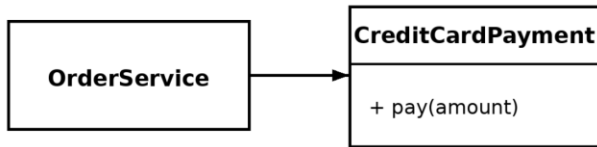
- It is one of the simplest of the principles and one of the hardest to get right.
- Finding and separating those responsibilities $\approx$ designing

Open/closed principle

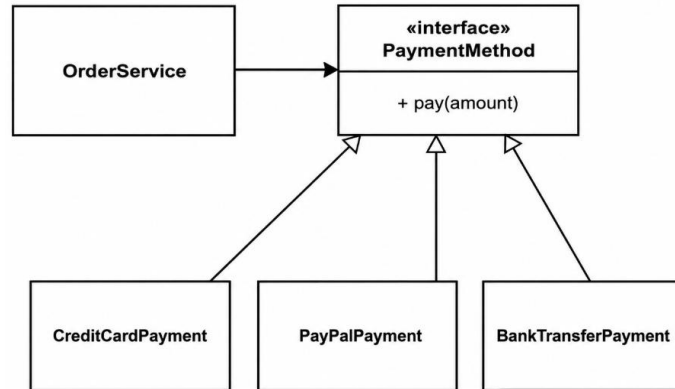
- ☾ Software entities (classes, modules, functions, ...) should be **open for extension** but **closed for modification**.
- ☾ Violation symptom: A single change results in a cascade of changes to dependent modules.

Open/closed principle

NOT OPEN AND CLOSED



OPEN AND CLOSED



Open/closed principle

Suggestion:

- ☾ In general, no matter how “closed” a module is, there will always be some kind of changes.
- ☾ It must be strategic
 - ▶ Choose the kinds of changes to prevent (after guessing)
 - ▶ Construct abstractions to protect against those changes

Open/closed principle

OCP is expensive

- ☾ It takes time and effort to create appropriate abstractions
 - increase the complexity of the design
- ☾ Apply OCP to changes that are likely to happen
 - To keep away from needless complexity
- ☾ What kind of changes are likely?
 - Guess, research, or wait until the changes happen, then redesign to prevent further changes of that kind.

Good Practice

- ☾ Make all member variables private. We expect that any other class, including subclasses are *closed* against changes in those variables.
- ☾ No global variables **ever**
- ☾ Run time type identification (instance of) is dangerous.

Conclusion on OCP

- OCP is at the heart of OOP design
- Conformance to this principle:
reusability and maintainability

Liskov substitution principle

- Subclass objects must be usable wherever superclass objects are expected.
- A subclass should not break the behavior promised by the superclass.
- Inheritance should preserve expected behavior, not only structure.
- Avoid subclasses that override methods with errors or unexpected restrictions.
- If a subclass cannot support a superclass method, the abstraction is probably wrong.

Barbara Liskov

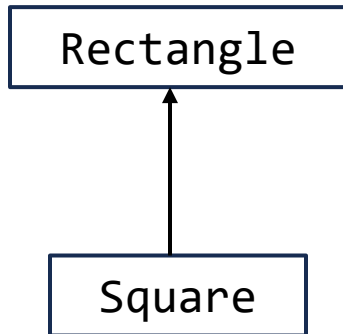


Liskov in 2010

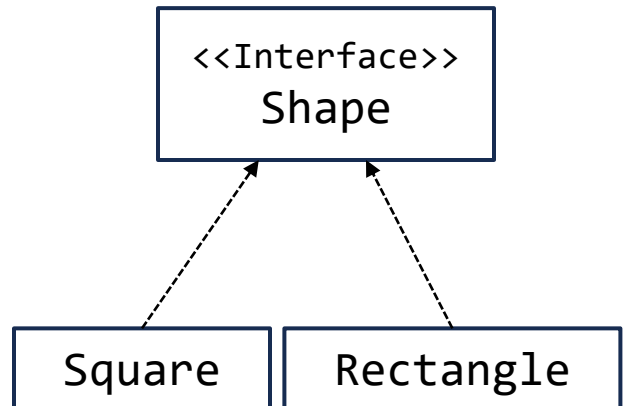
Source image: <https://www.wikipedia.org>

Liskov substitution principle

Example - violation of LSP



Solved violation by LSP



Conclusion on LSP

- ☾ LSP ensures that subclasses can safely replace their superclasses without changing the correctness of the program. It helps us design inheritance based on expected behavior, not only on conceptual “is-a” relationships.
- ☾ A good LSP-based design ensures that subclasses extend the behavior of the superclass rather than break it. This makes the system more reliable, reusable, and easier to maintain

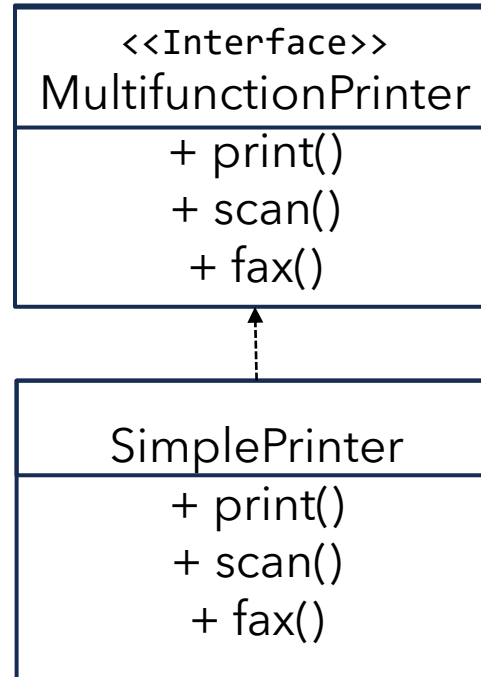
The Interface Segregation Principle

- ☾ Client should not be forced to depend upon interfaces that they do not use.
- ☾ To deal with “fat” interfaces, which can be broken up into groups of methods.
- ☾ Each group serves a different set of clients.

Segregate: keep one thing separate from another

The Interface Segregation Principle

Example – violation of ISP:



The Interface Segregation Principle

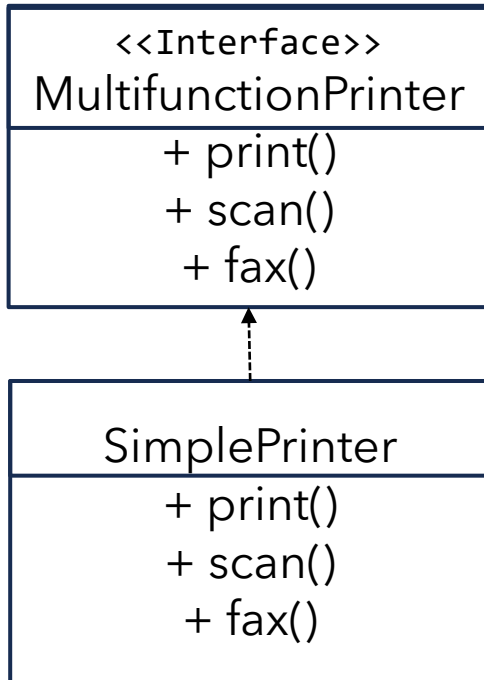
The “fat” interface

★ The interface of MultifunctionPrinter has been polluted with a method that it does not required. The added method is solely for the benefit of one of its subclass.

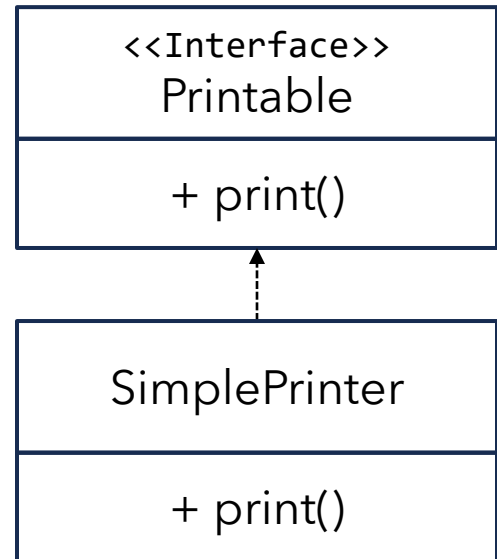
→ Pursing that design, for the needs of many subclasses, the interface of the MultifunctionPrinter become “fat”.

The Interface Segregation Principle

Example - violation of ISP



Solved violation by ISP



Dependency-Inversion Principle

- ☾ High-level modules should not depend on low-level modules.
- ☾ Both should depend on abstractions.
- ☾ Abstractions should not depend on details.
- ☾ Details should depend on abstractions.
- ☾ Use interfaces or abstract classes to decouple business logic from concrete implementations.
- ☾ Dependency Injection is a common technique to apply DIP.

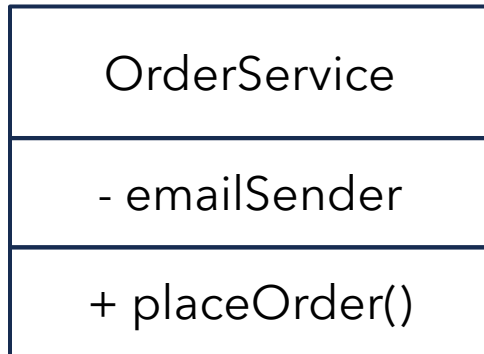
Dependency-Inversion Principle

- ⌚ High-level modules should not depend on low-level modules. Both should depend on abstractions.
- ⌚ Abstractions should not depend upon details. Details should depend upon abstractions.

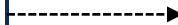
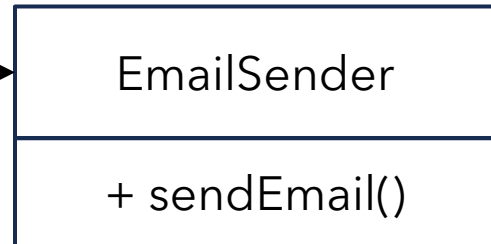
Dependency-Inversion Principle

Example of violation:

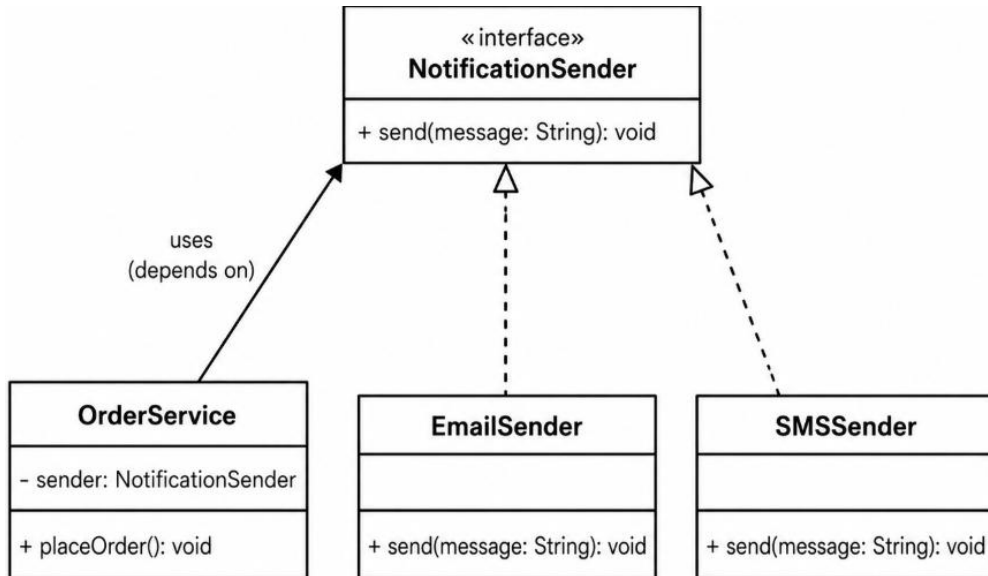
High-level module



Low-level module



Re-model violation with DIP



High-level module depends on abstraction, not concrete classes.

Conclusion on DIP

Ownership inversion:

- Don't call us; we'll call you
- The lower-level modules provides the implementation for interfaces that are declared within the upper-level modules.
- Dependence on Abstractions
- A class/a client should not depend on a concrete class, and rather on an abstract class or an interface

A class/a client should not depend on a concrete class, and rather on an abstract class or an interface

Key Takeaways

- ❑ **SRP - Single Responsibility Principle**

A class should have one clear responsibility and one main reason to change.

- ❑ **OCP - Open/Closed Principle**

Software entities should be open for extension but closed for modification.

- ❑ **LSP - Liskov Substitution Principle**

A subclass must be safely replaceable for its superclass without breaking program correctness.

- ❑ **ISP - Interface Segregation Principle**

A class should not be forced to implement methods it does not use.

- ❑ **DIP - Dependency Inversion Principle**

High-level modules should depend on abstractions, not concrete low-level implementations.